Arogyasathi — Complete System Architecture

Oct 17, 2026 · @Prudhvi

Project Overview

Arogyasathi is a production-grade frontline health-worker assistant for ASHA and ANM workers operating in rural India under patchy or zero connectivity. It answers clinical questions strictly from official protocol documents, detects danger signs before any model runs, and generates a prioritised daily visit list from the household register — working fully on a low-end Android phone with no internet, or routing to a stronger server model when connectivity is available. A parallel voice-call path via Twilio serves workers on keypad phones with no smartphone at all.

Core design principles:

- Safety rules fire before Laya or any LLM — the model is never in the critical path for a life-threatening decision
- Three fully independent access paths cover every connectivity scenario a rural ASHA worker faces
- Laya classifies query intent in 33 ms on-device CPU — zero GPU required
- Every answer cites the exact protocol section — zero hallucination on clinical facts by design
- Queries collected offline are reviewed and used to fine-tune Laya, delivered back via app update

Three Operating Modes

The app checks internet connectivity on every request and routes accordingly. No manual switching — the worker just asks her question.

Mode	Trigger	Device	Processing location	Model quality
Online	Good internet detected	Smartphone	Server (FastAPI)	Jev + strong LLM
Offline	No internet	Smartphone	On-device	Laya + Phi-3 Mini

Mode	Trigger	Device	Processing location	Model quality
Call	Voice call to IVR number	Any phone (keypad or smartphone)	Server via Twilio	Jev + strong LLM

Why three modes matter: 2G voice call coverage in rural India is significantly wider than 2G data coverage. A worker without data connectivity — or without a smartphone at all — can still reach the strongest model by dialling a number and entering her ASHA ID. The offline mode ensures that even total network failure never leaves her without answers.

Priority logic: Online mode is preferred whenever internet is available, because it offloads processing from the phone and delivers a stronger model. Offline mode is the fallback when connectivity drops. Call mode is always available in parallel, independent of the app.

RBAC Roles

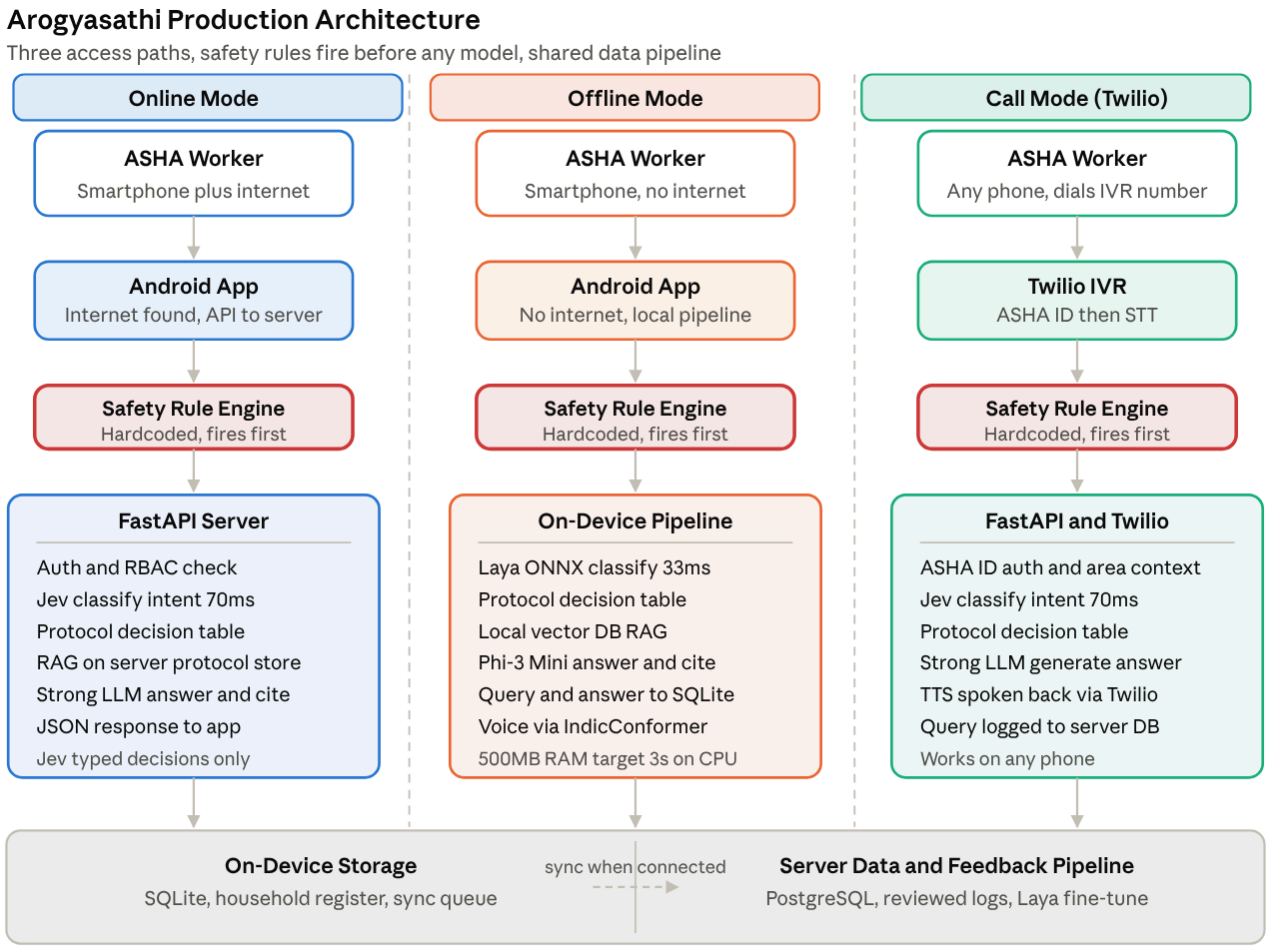
Four roles, enforced at the server. In call mode, the worker's ASHA ID authenticates her automatically on dial-in.

Role	Who	What they can do	What they cannot do
ASHA / ANM Worker	Frontline paramedical staff	Ask questions, view own household register, generate own daily visit list, record visit outcomes, trigger danger-sign escalation	See other workers' data, modify protocols, approve model updates
Supervisor (PHC Staff)	Primary Health Centre staff	View all workers' visit logs in their area, receive escalation alerts, review feedback queue before it reaches fine-tuning pipeline	Modify protocols, approve or deploy model updates
Medical Officer	Doctors receiving referrals	Receive structured referral summaries generated by the app, validate or correct protocol decisions, flag wrong answers for the improvement pipeline	Access raw worker logs, modify protocols

Role	Who	What they can do	What they cannot do
Admin	Central health tech team	Full access — manage user accounts, update protocol corpus, review and approve Laya model updates, push new model via app update	Nothing

How ASHA ID works in call mode: Worker dials the Twilio number → IVR prompts for ASHA ID → server authenticates against the worker database → pulls her household register and area context → personalises the answer to her assigned zone → logs the query against her ID for the feedback pipeline.

Complete System Architecture



three access modes, shared safety layer, and data pipeline

If the safety rule engine detects a danger sign in any mode, it outputs a refer-immediately alert and no model runs. The on-device storage syncs to the server whenever connectivity returns. Reviewed queries feed the Laya fine-tuning pipeline, and the updated model is distributed to field devices as an app update.

Mode 1 — Online Mode

Triggered when the Android app detects an active internet connection at request time.

1. **Connectivity check** — app pings a lightweight health endpoint on the FastAPI server. If response time is under 800 ms, online mode is selected.
2. **Input** — worker types or speaks a query. Voice input goes through IndicConformer on-device (offline STT, 39 MB) to produce query text before sending.
3. **HTTPS request to FastAPI server** — query text, worker's ASHA ID, and app version sent as a signed JSON payload.
4. **Auth + RBAC** — server validates the ASHA ID against the worker database, retrieves the worker's assigned zone and household register context, and confirms the role is ASHA or ANM.
5. **Safety rule engine (server-side)** — identical rule set to the on-device engine. Any matched danger sign returns a 200 with `{"action": "refer_immediately", "reason": "<matched rule>"}` and the pipeline stops.
6. **Jev classification (70 ms)** — query text fed to Jev on the server. Returns a typed structured output: `{protocol_category, confidence, is_out_of_scope}`. No hallucination possible on this step.
7. **Protocol decision table lookup** — category maps to an approved action and the exact protocol section. This lookup is deterministic — no model generates the clinical action.
8. **Protocol RAG** — server-side vector store retrieves the top-k relevant chunks from the official protocol corpus using the category and query as the retrieval signal.
9. **Strong LLM generation** — LLM receives the approved action, retrieved chunks, and query. It generates a plain-language explanation and appends the protocol section citation. It cannot override the approved action.
10. **Response to app** — JSON payload with answer text, citation, confidence score, and protocol category. App renders the answer on screen.
11. **Logging** — query, classification result, and answer logged to the server DB against the worker's ASHA ID for the feedback pipeline.

Fallback: If the server returns an error or the connection drops mid-request, the app automatically falls back to offline mode for that query and notifies the worker.

Mode 2 — Offline Mode

Triggered when no internet is detected. The entire pipeline runs on the Android device with no network calls.

1. **Connectivity check fails** — health endpoint unreachable or response over 800 ms. App switches to offline pipeline silently.
2. **Input** — worker types or speaks. Voice goes through IndicConformer (on-device, offline) producing query text.
3. **Safety rule engine (on-device)** — hardcoded rule set in a local JSON file. Pattern-matched against the query text in under 10 ms. If a danger sign rule fires, the app immediately displays the refer-immediately alert. No further steps run.
4. **Laya ONNX classification (33 ms on CPU)** — query text fed to the quantised Laya model running via ONNX Runtime on the device CPU. Returns `{protocol_category, confidence, is_out_of_scope}` as typed structured output.
5. **Out-of-scope check** — if `is_out_of_scope` is true, the app displays a fixed refusal message: "This is outside my protocols. Please consult a doctor or supervisor." Pipeline stops.
6. **Protocol decision table lookup (local)** — same deterministic table as the server, stored as a local JSON file. Category maps to approved action and protocol section reference.
7. **Local vector DB retrieval** — query embedded using the pre-cached small embedding model. Top-k chunks retrieved from the local vector store (SQLite-based, built from the protocol corpus at install time).
8. **Phi-3 Mini / Gemma 2B generation** — 4-bit quantised small LLM receives the approved action, retrieved chunks, and query. Generates a plain-language explanation with the protocol section citation. Target response time: under 3 seconds on a low-end CPU.
9. **Answer displayed** — rendered on screen with the protocol citation visible.
10. **SQLite logging** — query text, Laya classification result, answer, and timestamp written to local SQLite under the worker's ASHA ID.
11. **Sync queue** — the logged entry is added to a pending sync queue. When internet connectivity is next detected, entries are uploaded to the server DB in a background job and removed from the queue.

Memory budget: Laya ONNX model under 150 MB, embedding model under 80 MB, Phi-3 Mini INT4 under 2.5 GB, local vector DB under 50 MB. Total active footprint target: under 500 MB RAM during inference.

Mode 3 — Call Mode (Twilio)

Triggered when the worker dials the dedicated Twilio number from any phone — keypad phone, feature phone, or smartphone. No app, no internet data required. Works on a voice-only 2G connection.

1. **Worker dials the number** — Twilio receives the call and initiates an IVR (Interactive Voice Response) session.
2. **ASHA ID entry** — IVR prompts the worker to enter her ASHA ID using the keypad (DTMF tones). This is the authentication step. Twilio forwards the DTMF input to the FastAPI server.
3. **Server auth + RBAC** — FastAPI validates the ASHA ID, retrieves the worker's assigned zone, area context, and household register from the server DB. If the ID is invalid, the IVR speaks an error message and the call ends.
4. **Voice query** — IVR prompts: "Please speak your question now." The worker speaks her query in Telugu or Hindi.
5. **STT — IndicConformer (server-side)** — Twilio streams the audio to the FastAPI server. IndicConformer transcribes the speech to text. For critical medical terms, transcription confidence below 0.7 triggers a re-prompt: "I did not understand. Please repeat."
6. **Safety rule engine (server-side)** — transcribed query text matched against the hardcoded danger sign rules. If matched, Twilio immediately speaks the alert in Telugu: "Emergency — refer to hospital immediately." Call ends after confirmation.
7. **Jev classification (70 ms)** — transcribed query fed to Jev. Returns typed structured output: `{protocol_category, confidence, is_out_of_scope}` with calibrated probabilities.
8. **Out-of-scope check** — if flagged, Twilio speaks: "This is outside my protocols. Please consult your supervisor or the nearest PHC doctor." Call ends.
9. **Protocol decision table lookup** — deterministic lookup of the approved action and protocol section.
10. **Protocol RAG + Strong LLM** — same server-side pipeline as online mode. Generates a concise plain-language answer optimised for spoken delivery (short sentences, no markdown).
11. **TTS — spoken back to worker** — answer converted to speech in Telugu using the server-side TTS engine. Twilio plays the audio to the worker. The answer includes the protocol section reference spoken aloud.
12. **Call log** — query text, classification, answer, and call metadata (duration, ASHA ID, timestamp) logged to server DB.

13. **Feedback flag** — worker can press 1 (helpful) or 2 (not helpful) at the end of the call. This flag is stored alongside the log for the feedback pipeline review.

Why Twilio: Twilio's telephony API handles call routing, DTMF parsing, audio streaming, and TTS/STT integration with a single webhook-based architecture that the FastAPI server can respond to with TwiML. This avoids building a custom telephony stack and works in India with local number provisioning.

Full Tech Stack

Layer	Component	Tool / Model	Reason
On-device classification	Query intent router	Laya (ONNX Runtime, CPU)	33 ms, 100+ languages, Apache 2.0, fine-tuneable
On-device generation	Answer generator	Phi-3 Mini INT4 or Gemma 2B INT4	Runs on CPU under 3 s, fits in 500 MB RAM
On-device ASR	Voice input	IndicConformer (AI4Bharat, ONNX)	Telugu-first, tested on budget Android, fully offline
On-device storage	Household register + logs	SQLite	Zero dependency, works offline, fast queries
On-device retrieval	Protocol RAG	Small embedding model + SQLite vector index	Offline, no external DB needed
Server classification	Query intent router (online + call)	Jev (TypeSafe AI)	70 ms, typed outputs, zero hallucination on routing
Server generation	Answer generator (online + call)	Strong LLM (GPT-4 class or open equivalent)	Higher quality than on-device, used only when connected
Server ASR	Voice transcription (call mode)	IndicConformer (server-side)	Same model as on-device, consistent Telugu accuracy
Server TTS	Spoken answer (call mode)	IndicTTS or native Android TTS via Twilio	Telugu voice output for call mode

Layer	Component	Tool / Model	Reason
Server framework	API + webhook handler	FastAPI (Python)	Async, fast, easy Twilio webhook integration
Telephony	IVR + voice call routing	Twilio (TwiML + streaming)	India number provisioning, DTMF, audio streaming
Server DB	Worker accounts + logs + protocol store	PostgreSQL	Reliable, supports vector extensions if needed
Feedback pipeline	Reviewed log curation + Laya fine-tune	Python pipeline + Laya fine-tuning API	Controlled improvement loop, not automatic
App platform	Android app	Kotlin + ONNX Runtime Android	Low-end device support, ONNX native integration
Safety rules	Danger sign detection	Hardcoded JSON rule set	Deterministic, 100% recall by design, no model dependency
Protocol decision table	Approved action lookup	Local JSON (on-device) + PostgreSQL (server)	Deterministic, version-controlled, clinician-approved
Auth	Worker identity + RBAC	ASHA ID + JWT + role table in PostgreSQL	Simple, works in call mode via DTMF
Sync	Offline queue to server	Background service + REST endpoint	Lightweight, duplicate-safe, conflict-handled

What Each Teammate Builds

Three roles map cleanly to the three layers of the system. Split by who owns what, not by mode — each person works across all three modes for their layer.

Role A — Safety Engineer + Protocol Curator

This is the most critical role. If the safety rules fail, everything fails.

- Write every hardcoded danger sign rule in the JSON rule set (bleeding in third trimester, newborn not breathing, severe dehydration, eclampsia signs, and all others from the H5 protocol corpus)

- Build the protocol decision table mapping each Laya category to an approved action and protocol section reference
- Verify 100% recall on the hackathon's escalation test set before any other component is added
- Set up the protocol corpus: chunk the official documents, build the local vector index, and load it into the server RAG store
- Write the out-of-scope refusal rules
- Own the evaluate.py test run and fix any failures

Role B — Android Developer

- Build the Android app in Kotlin
- Integrate Laya via ONNX Runtime Android for on-device classification
- Integrate IndicConformer for offline voice input
- Integrate Phi-3 Mini or Gemma 2B INT4 for on-device generation
- Build the connectivity check and mode-switching logic
- Build the SQLite schema: household register, visit log, sync queue
- Build the daily visit list view from the household register
- Build the offline sync queue and background sync job
- Measure and report: model size, peak RAM, cold-start time, response time on the target device

Role C — Server + ML Engineer

- Build the FastAPI server: auth endpoint, RBAC middleware, classification route, generation route, Twilio webhook handler
- Integrate Jev for server-side classification (online and call modes)
- Integrate the strong LLM for server-side generation
- Set up Twilio: provision an Indian number, configure the IVR TwiML, wire DTMF auth, connect STT and TTS
- Build the feedback collection pipeline: receive synced logs, store reviewed labels, run Laya fine-tuning, evaluate the updated model, distribute via app update
- Set up PostgreSQL: worker table, protocol store, query log table, feedback label table
- Build the ASHA ID authentication and JWT issuance

Shared responsibilities (everyone)

- Test danger sign rules against the full escalation set before submission
- Run evaluate.py and fix failures together
- Prepare the demo: offline mode in airplane mode, danger sign trigger, daily visit list, call mode walkthrough

- Write the one-paragraph winning proposal statement for the PPT